

***Project 1***

*(Due: Sept 20, 2026)*

*The project will be done in pairs. By Sept 14, please make sure that you have a partner to work with. If not, please send me a message.*

Two fundamental uninformed search techniques are Depth-first-search (DFS) and Breadth-first-search (BFS). In this project, you will be implementing the tree search versions of basic uninformed search algorithms. We have seen that BFS is guaranteed to terminate (if the state-space graph is finite) and find an optimal solution (in the unweighted case), but its memory requirement is exponential in the depth of search. DFS may not find an optimal solution even in the unweighted case, but its far less storage (linear in the depth of search). Both take time that is exponential in the depth of search (in the worst-case). In this project, you shall implement and compare BFS with a variant of DFS called DLS (depth-limited search) and IDS (iterative deepening search) which uses DLS repeatedly to find an optimal solution.

The problem we shall solve involves solving a widely known puzzle called the sliding tile puzzle, shown below. The puzzle involves  $n^2$  tiles in a square board numbered 1 to  $n^2 - 1$  with an empty square to which one can slide an adjacent tile in U, D, L or R sliding move.



Given a starting state  $s$  and a goal state  $g$ , your program will find the shortest move sequence to reach  $g$  from  $s$ , and print the move sequence. It should also output the total number of nodes expanded (removed from the queue), the maximum size of the queue and the CPU time taken. The methods to be implemented are BFS, DLS and IDS. In each case, we are using tree search approach which means a state may enter the queue or stack even if it has already entered before. The only reentry we will avoid is the entry of a parent node as a successor of a child node.

As we have noted, DFS may fail to find the best solution, and it may loop forever since in the tree search version. So instead of DFS, we will implement DLS, the depth-limited version of DFS which is essentially DFS performed to specified depth  $L$ . Thus, DLS takes as input a start state, a goal state and a depth  $L$ . It will search for a solution to depth  $L$  (the root is considered to be at depth 0) and report a solution if it exists, else returns failure. DLS forms the basis for IDS which calls DLS for depth set to 0, 1, 2, until a solution is found.

Your choice of data structures should be to make the algorithm perform efficiently but it should also be simple so that testing and debugging can be done effectively and in a reasonable time frame.

Input/output format: The input will be represented as a vector of length 4 each member in turn a vector of length 4. 0 will be used for the empty slot. For example, the above board will be represented as

[ [ 1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [13, 14, 15, 0] ].

Both DFS and IDS shall be implemented using an explicit stack. BFS shall be implemented using a queue.

Your UI should be such that it asks the user to enter the start state in the above format, then it asks for the goal state (again in the same format). Then it asks the user to choose the algorithm to run: (1) BFS (2) IDS (3) BFS and IDS (4) DLS

Output format will be:

The solution found (or not found).

The number of moves: \_\_\_\_\_

The sequence of moves: (as a string over the alphabet {L, R, U, D})

The number of states removed: \_\_\_\_\_

The maximum size of the queue (stack): \_\_\_\_\_

CPU time: \_\_\_\_\_

Some test cases will be provided soon.

What should be submitted?

- 1) A program that provides the functionality with the user options described above.
- 2) A transcript of the dialog with an AI assistant, if used.
- 3) A one to two page description of the data structures used and the reasons for the choices.

The write-up shall also include a comparison between the theoretical bound on the time and space complexity presented in class and what is observed in your implementation. Include at least one test case for each of BFS, DLS and IDS.

The report (3) should be done without any support by the AI assistant.